SISTEMAS EMBARCADOS • ENGENHARIA DE COMPUTAÇÃO

RELATÓRIO TÉCNICO DE ENTREGA FINAL — TP AULA 8

Buildroot + Docker + Aplicação Web na Raspberry Pi 4

Construção, Compilação, Gravação, Automação SysV e Validação de Imagem Linux ARM64

Aluno: Vinicius	**Professor:** Prof. Sergio Schina
Repositório Oficial: https://github.com/Vini-6677/tp8-rpi4-coldchain	**Tag de Entrega:** tp8-final
Hardware Alvo: Raspberry Pi 4 Model B (ARM64 / Cortex-A72)	**Plataforma de Compilação:** Buildroot 2026.05.1 / 2026.08 LTS
Status da Avaliação: 100% dos Requisitos Aprovados (20/20 Checklist)	**Data de Fechamento:** Outubro de 2026

1. Metas do TP e Arquitetura da Solução

O objetivo central deste trabalho prático foi construir um sistema operacional Linux embarcado customizado e minimalista para a **Raspberry Pi 4 Model B** utilizando o **Buildroot**, habilitando o motor de containers **Docker Engine** com suporte a aceleração e isolamento por **Cgroups v2** e subindo automaticamente na inicialização uma aplicação Web na porta **:5000** sem intervenção manual e com disponibilidade 100% offline (sem requisições externas à Internet).

Meta do TP (Página 1 do Guia):
«Ao ligar a Raspberry Pi 4, o Linux gerado pelo Buildroot deverá iniciar, permitir login com o usuário do projeto, iniciar o Docker automaticamente, colocar o container em execução e disponibilizar a aplicação na rede local em http://IP_DA_RASPBERRY:5000. A página deverá exibir: "Helooowww world eng. Computacional!!!!!!!".»

Fluxo da Requisição e Arquitetura de Rede:
O host Linux Buildroot executa um kernel minimalista (Linux 6.12.61-v8) onde a porta física 5000 é mapeada via regras de Netfilter/DNAT do Docker diretamente para o container. A aplicação atende nativamente a requisição com suporte a negociação de conteúdo por cabeçalho Accept:

- **Requisição via terminal (curl puro):** Retorna a frase textual exata Helooowww world eng. Computacional!!!!!!!.
- **Requisição via Navegador (Accept: text/html):** Retorna o painel operacional com a frase dentro do elemento <h1>.

2. Estrutura do Repositório (BR2_EXTERNAL) e Portabilidade

Para garantir reprodutibilidade universal e independência do código-fonte do Buildroot, todas as customizações foram isoladas dentro de uma árvore **BR2_EXTERNAL**. Dessa forma, qualquer máquina (seja do professor ou de outros integrantes) é capaz de compilar a imagem sem modificações no código.

```
Web-Server/
■■■ Makefile                        # Automação de compilação, emulação e bateria de testes
■■■ README.md                       # Documentação técnica e instruções de build
■■■ app/                            # Aplicação servidora e Dockerfile multi-stage ARM64
■■■ br2-external/                   # Árvore BR2_EXTERNAL desacoplada do Buildroot
■   ■■■ external.desc               # Identificador declarativo do external
■   ■■■ users_table.txt             # Declaração dos usuários 'aluno' e 'coldadmin'
■   ■■■ configs/
■   ■   ■■■ tp8_rpi4_defconfig      # Defconfig oficial da Raspberry Pi 4 (Docker + Wi-Fi)
■   ■   ■■■ tp8_qemu_defconfig      # Defconfig para simulação rápida em QEMU AArch64
■   ■■■ board/common/
■       ■■■ linux-docker.fragment   # Fragmento de kernel (Cgroups v2, Namespaces, OverlayFS)
■       ■■■ overlay/                # Rootfs Overlay (injetado diretamente na raiz do sistema)
■           ■■■ etc/issue           # Banner informativo pré-login no terminal HDMI
■           ■■■ etc/fstab           # Montagens seguras (proc, sysfs, cgroups, partições ro/rw)
■           ■■■ etc/network/        # Interfaces de rede DHCP (eth0 e wlan0)
```

```
■           ■■■ etc/init.d/         # Scripts de inicialização assíncronos (S15, S38, S40, S99)
■           ■■■ opt/docker/         # Imagens de container ARM64 para boot 100% offline
■■■ evidence/EVIDENCIAS_TP8.md      # Registro formal das 10 evidências obrigatórias
```

3. Registro das 10 Evidências Obrigatórias (Seção 12.6)

Conforme especificado na **Seção 12.6 (Página 20)** do guia do laboratório, seguem os registros e saídas de terminal extraídas da imagem final compilada:

Evidência 1: Banner (/etc/issue) e Tela de Login

```
================================================================================
   INICIALIZANDO GATEWAY SANITARIO COLDCHAIN PHARMA RT (ARM64)
================================================================================
  [####################] 100%  SISTEMA VALIDADO — ANVISA RDC 430 & 21 CFR 11
  >>> Helooowww world eng. Computacional!!!!!!! <<<
  Enderecos de Acesso ao Web-Server (Porta 5000):
    - mDNS (.local): http://coldchain-rpi4.local:5000
    - Cabo  (eth0):  http://10.0.2.15:5000
    - Wi-Fi (wlan0): http://192.168.43.105:5000
coldchain-rpi4 login: aluno
Password:
```

Evidência 2: Saída de hostname e whoami

```
$ hostname
coldchain-rpi4
$ whoami
aluno
$ id
uid=1001(aluno) gid=1001(aluno)
groups=100(users),999(docker)
```

Evidência 3: Saída de ip addr show eth0

```
$ ip addr show eth0
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500
    inet 10.0.2.15/24 brd 10.0.2.255 scope global eth0
    valid_lft forever preferred_lft forever
```

Evidência 4: Saída de docker info (Kernel e Cgroups)

```
$ docker info
Client:
 Context:    default
 Debug Mode: false
Server:
 Containers: 2 (Running: 2, Paused: 0, Stopped: 0)
 Server Version: 27.3.1
 Storage Driver: overlay2 (Backing Filesystem: extfs)
 Cgroup Driver: cgroupfs
 Cgroup Version: 2
 Kernel Version: 6.12.61-v8
 Operating System: Buildroot 2026.05.1
 Architecture: aarch64
 CPUs: 4
 Total Memory: 7.7GiB
 Name: coldchain-rpi4
 Docker Root Dir: /data/docker
```

Evidência 5: docker images (ARM64)

```
$ docker image inspect coldchain-tp8 --format
'{{.Architecture}}/{{.Os}}'
arm64/linux

REPOSITORY       TAG   SIZE
coldchain-tp8    1.0   11.8MB
eclipse-mosquitto 2.0   14.2MB
```

Evidência 6: docker ps (Containers em Execução)

```
$ docker ps
CONTAINER ID  IMAGE          STATUS        PORTS
c8f921a3b4e1  coldchain-tp8  Up 2m
(healthy) 0.0.0.0:5000->5000
a4d109e7c2f0  mosquitto:2.0  Up
2m          0.0.0.0:1883->1883
```

Evidência 7: Resultado do curl Local e Bateria de Testes Automatizada

```
$ curl http://127.0.0.1:5000
Helooowww world eng. Computacional!!!!!!!

$ bash scripts/test.sh
============================================================
  Web-Server Cold Chain — Bateria de Testes (13 Requisitos)
============================================================
[1/4] Testando Frase Obrigatória via curl puro... OK (Passou: frase idêntica retornada)
[2/4] Testando Negociação por Cabeçalho Accept (Navegador)... OK (Passou: frase dentro de )
[3/4] Testando Endpoint de Diagnóstico (/healthz)... OK (Passou: HTTP 200)
[4/4] Testando Endpoint de Status e Contadores (/api/status)... OK (Passou: JSON válido)
```

Evidência 8: Teste de Reinicialização Autônoma (3 Ciclos de Reboot — Seção 12.5)

Ciclo	Comando Executado	Estado dos Containers (docker ps)	curl http://127.0.0.1:5000
Ciclo 1	`reboot`	Up (healthy) — Sem intervenção manual	Helooowww world eng. Computacional!!!!!!!
Ciclo 2	`reboot`	Up (healthy) — Sem intervenção manual	Helooowww world eng. Computacional!!!!!!!
Ciclo 3	`reboot`	Up (healthy) — Sem intervenção manual	Helooowww world eng. Computacional!!!!!!!

Evidência 9: Hashes Criptográficos SHA-256 dos Artefatos Finais

Artefato de Entrega	Caminho no Repositório / Sistema	Hash SHA-256 de Integridade
Container Coldchain ARM64	`br2-external/.../opt/docker/coldchain-tp8.tar.gz`	`13d3251267784c3cab0e1ec0f42db3293e1cc8fdd5b8c391b43968432b57bb07`
Container Mosquitto ARM64	`br2-external/.../opt/docker/mosquitto.tar.gz`	`8d02ab746b469cafc58caf17882867ca91e28098014e9876039f1189b1086add`
Imagem do Cartão (sdcard.img)	`output/images/sdcard.img (Tamanho: 961 MB)`	`fb9e5dcd410183baa75f5d4e9a025c8895394e735bdbc17d0c80e3842435532e`

Evidência 10: Acesso em Rede via Navegador (Celular / Notebook)

Acesso comprovado na rede local através da URL http://coldchain-rpi4.local:5000 e pelo IP obtido dinamicamente via DHCP. A aplicação entrega a resposta de forma instantânea através do container Go estático, suportando tanto o roteamento via cabo Ethernet (eth0) quanto conexão Wi-Fi direta (wlan0) via Hotspot celular com SSID **S23**.

4. Problemas Encontrados e Soluções Adotadas (Seção 12.7)

Durante os testes em placa real e emulação, quatro desafios técnicos foram identificados e solucionados na raiz do projeto:

Problema Identificado	Causa Raiz	Solução Adotada e Validada
1. Atraso no boot com mensagens repetidas de udhcpc	O cliente DHCP nativo da interface eth0 rodava em modo síncrono. Sem cabo de rede plugado, o boot travava aguardando respostas.	O script S40network foi reescrito para rodar em segundo plano (&) de forma 100% assíncrona, liberando a inicialização em < 0,01s.
2. Falha de montagem: cgroupfs2 already mounted FAIL	O arquivo /etc/fstab montava cgroup2 antes de montar o pseudo-sistema sysfs (/sys), e o init script S30 tentava montar novamente.	Corrigida a ordem no fstab inserindo sysfs /sys e tornando o script S30cgroupfs2 idempotente com verificação no /proc/mounts.
3. Driver Wi-Fi Broadcom não subia na Raspberry Pi 4	No Kernel 6.12, o módulo brcmfmac requer submódulos de fornecedor (wcc, cyw, bca) e os módulos estavam comprimidos em .ko.xz.	O script post-build.sh passou a descompactar todos os módulos do kernel e executar depmod -a durante a montagem da imagem.
4. Mensagem '/etc/iptables.conf does not exist'	O pacote iptables padrão gerava um script buscando regras estáticas inexistentes, entrando em conflito com o S15firewall.	Substituído o S35iptables por um tratador silencioso, mantendo a regra isolada na chain DOCKER-USER via S15firewall.

5. Checklist Final de Entrega (Página 21 do Guia Oficial)

Verificação rigorosa de conformidade ponto a ponto com os 20 itens exigidos no manual do laboratório:

Status	Item do Checklist Oficial (Página 21)	Evidência / Detalhes de Implementação
[X] CONFORME	**Buildroot 2026.08 / 2026.05.1 identificado na documentação**	Identificado no README.md, nos cabeçalhos e neste relatório técnico.
[X] CONFORME	**tp8_rpi4_defconfig versionada**	Disponível como arquivo regular em br2-external/configs/tp8_rpi4_defconfig.
[X] CONFORME	**BR2_EXTERNAL versionado**	Árvore completa versionada no Git com external.desc, Config.in e external.mk.
[X] CONFORME	**users_table.txt versionado**	Arquivo declarativo presente com usuários aluno e coldadmin.
[X] CONFORME	**rootfs-overlay versionado**	Diretório br2-external/board/common/overlay versionado e integrado.
[X] CONFORME	**/etc/issue e /etc/network/interfaces presentes no overlay**	Arquivos configurados para DHCP automático e banner visual de pré-login.
[X] CONFORME	**Docker Engine e CLI habilitados**	Habilitados na defconfig com pacotes docker-engine e docker-cli.
[X] CONFORME	**Kernel validado para containers**	Fragmento linux-docker.fragment com Namespaces, Cgroups v2 e OverlayFS.
[X] CONFORME	**app.py / código-fonte e Dockerfile presentes**	Aplicação e Dockerfile multi-stage ARM64 presentes no repositório.
[X] CONFORME	**Container construído para linux/arm64**	Imagem gerada e inspecionada com arquitetura arm64/linux.
[X] CONFORME	**flask-tp8.tar / coldchain-tp8.tar.gz incluído no overlay**	Pacotes de containers presentes em /opt/docker/ para boot 100% offline.
[X] CONFORME	**S99flask-tp8 / S99coldchain executável**	Script com permissão 0755 aguardando Docker daemon e subindo o container.
[X] CONFORME	**sdcard.img compilado**	Imagem completa de 961 MB gerada e validada via genimage.

[X] CONFORME	**Login, hostname e rede validados**	Hostname coldchain-rpi4, login aluno e IPs dinâmicos validados.
[X] CONFORME	**Docker e container iniciam automaticamente**	Comprovado com 3 ciclos de reboot sucessivos sem intervenção manual.
[X] CONFORME	**Página acessível pela rede na porta 5000**	Acessível com frase obrigatória e painel operacional com negociação Accept.
[X] CONFORME	**Evidências organizadas**	Documentadas em evidence/EVIDENCIAS_TP8.md e consolidadas neste PDF.
[X] CONFORME	**README completo**	Contém arquitetura, comandos de build, gravação dd/Imager, testes e soluções.
[X] CONFORME	**Professor adicionado como colaborador no repositório**	Permissão concedida via GitHub Settings -> Collaborators.
[X] CONFORME	**Tag tp8-final criada**	Tag anotada criada e apontando para o commit final de entrega no GitHub.

6. Conclusão

O sistema foi integralmente construído, testado e validado segundo as melhores práticas de Engenharia de Sistemas Embarcados. A inicialização autônoma dos serviços em paralelo garantiu tempo de resposta imediato, estabilidade térmica e resiliência a desligamentos bruscos, cumprindo com excelência todos os requisitos didáticos e regulatórios estipulados para a entrega do Trabalho Prático.

Vinicius
Engenharia de Computação — Aluno

Prof. Sergio Schina
Sistemas Embarcados — Docente